SPRINT 3 CISIA - RÉVISION AELION 28/09/2026

Pas-à-pas PayGuard

Cette page remplace les anciens chemins ZIP et pack du guide REX joint.

Chemin dans l'extranet Dendreo / AELION

Concevoir et implémenter une solution d'IA > Public - Participants de l'ADF > Tous les Participants > ressources > Sprint 3

Sous-dossiers : 01_PRESENTATIONS_PPTX · 02_PRESENTATIONS_PDF · 03_GUIDES_FICHES_ET_REVISION · 04_FICHIERS_A_COMPLETER · 05_DONNEES_ET_EXERCICES · 06_Schéma_et_DAT · corrigé · fiches de révision autres sprints

Consignes mises à jour

• Télécharger les fichiers du dossier 05_DONNEES_ET_EXERCICES/TP_PAYGUARD ; il n'y a aucun ZIP à décompresser.

• Dans un dossier local court, recréer data, data/labels, scripts et tests à partir des noms contenant __ ; retirer le suffixe .txt des scripts, de pyproject.toml et de uv.lock. Le manifeste des exercices donne chaque nom et chaque SHA-256.

• La structure de départ doit contenir pyproject.toml, uv.lock, les jeux CSV, les scripts et les tests avant uv sync --frozen --extra dev. Le Markdown AELION daté du 28/09 contient l'étape 0 corrigée.

Les noms avec __ représentent un ancien séparateur de dossier. Les fichiers .py.txt, .toml.txt, .lock.txt, etc., conservent le contenu source ; retirez .txt après téléchargement. MODE_EMPLOI_FICHIERS_INDIVIDUELS.md et MANIFESTE_EXERCICES_INDIVIDUELS.json sont dans 05_DONNEES_ET_EXERCICES.

Empreinte SHA-256 du PDF REX d'origine : c68a04ce20c78f20c3f98d498ef2a8276991b09d98cd94319562d736d811c742

TP autonome - Data drift & métriques : l'affaire PayGuard

À retenir
TP d'application du module 31 (Data drift & métriques de dérive - concepts), indépendant d'InduSense : nouveau domaine, mêmes concepts. Vous êtes l'équipe data d'un prestataire de paiement - à vous de surveiller un modèle anti-fraude en production. = à produire par vous · = fourni. Le formateur annonce le rythme, les pauses et l'ouverture éventuelle du bonus.

0. Le scénario

PayGuard est le service anti-fraude d'un site e-commerce français. Un modèle de ML note chaque transaction carte bancaire : au-dessus d'un seuil gelé, la transaction part en vérification manuelle chez l'équipe risque.

- Une fraude ratée (FN) coûte cher : remboursement du client, litige → ≈ 50 €.
- Une vérification inutile (FP) coûte peu : quelques minutes d'analyste → ≈ 2 €.

Le modèle a été entraîné sur une période de référence (20 000 transactions labellisées). Depuis, il tourne en production. Vous recevez trois semaines de production à auditer (3 000 transactions chacune). Détail réaliste qui change tout : les labels (fraude confirmée ou non) arrivent avec plusieurs semaines de retard - le temps que les clients contestent et que les enquêtes aboutissent. Pendant les étapes 3 à 5, vous travaillez donc sans labels, comme en vrai.

Les 5 features du modèle (fichiers `data/*.csv`) :

Feature	Description
`montant_eur`	montant de la transaction
`heure`	heure de la transaction (0-24, décimale)
`ancienneté_client_j`	ancienneté du compte client, en jours
`nb_articles`	nombre d'articles du panier
`distance_domicile_km`	distance entre l'adresse de livraison et le domicile

Vos livrables en fin de TP :

1. `scripts/drift_lab.py` et `scripts/evaluate_semaine.py` complétés - `uv run python -m pytest` entièrement vert ;
2. les tables et figures de dérive dans `reports/` ;
3. `reports/drift_spec_payguard.md` : votre mini drift spec (gabarit fourni à l'étape 7) ;
4. vos réponses écrites aux questions (journal de bord ou fichier à part - elles se discutent en correction).

À retenir
⚠ Règles du jeu : ne modifiez jamais `tests/test_drift.py` (c'est votre « definition of done »). N'ouvrez pas `data/labels/` ni `scripts/evaluate_semaine.py` avant l'étape 6 - en production, ces labels n'existent pas encore. Et si un dossier `formateur/` traîne, il ne vous est pas destiné.

Étape 0 - Installation

Prérequis : Python 3.13 et, de préférence, uv (l'outil utilisé sur tout le parcours).

Décompressez le zip dans un chemin Windows court, par exemple `%USERPROFILE%\CISIA\S3\tp_autonome_drift`, puis ouvrez un terminal dans ce dossier.

Évitez `Téléchargements\...\plusieurs_sous_dossiers\...` : Windows peut alors annoncer un sync réussi tout en omettant une extension compilée de scikit-learn.

À retenir
Si `ModuleNotFoundError` cite un sous-module interne de `sklearn.metrics` après le sync : ne lancez ni `pip install` ni `uv add`. Fermez VS Code, redécompressez le zip dans le chemin court ci-dessus, rouvrez ce dossier dans VS Code, puis rejouez `uv sync --frozen --extra dev`.

Voie recommandée : uv

Windows (PowerShell) :

```
# si uv n'est pas déjà installé :
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
# fermez/rouvrez le terminal, puis :
$env:PYTHONUTF8 = "1"       # évite les erreurs d'affichage Unicode sous Windows
uv sync --frozen --extra dev
uv run python -c "import numpy, pandas, scipy, sklearn, matplotlib; print('setup OK')"
uv run python -m pytest -q
```

Sortie de départ attendue dans le zip apprenant : `10 failed · 1 passed · 1 skipped`. C'est normal : les dix échecs matérialisent les fonctions que vous allez compléter et le test ignoré attend l'artefact modèle produit plus tard. Ne modifiez jamais `tests/test_drift.py` pour « faire du vert ». À la fin du TP, la même commande doit afficher `12 passed`.

macOS ou Linux (terminal) :

```
# si uv n'est pas déjà installé :
curl -LsSf https://astral.sh/uv/install.sh | sh        # (ou : brew install uv)
# fermez/rouvrez le terminal, puis :
uv sync --frozen --extra dev
uv run python -c "import numpy, pandas, scipy, sklearn, matplotlib; print('setup OK')"
uv run python -m pytest -q
```

Sortie de départ attendue : `10 failed · 1 passed · 1 skipped` ; preuve finale : `12 passed`.

Voie de repli : venv + pip (si uv coince en salle)

Windows (PowerShell) :

```
py -3.13 -m venv .venv
.venv\Scripts\Activate.ps1     # si blocage : Set-ExecutionPolicy -Scope Process Bypass
pip install -r requirements.txt
@'
import sys
import numpy, pandas, scipy, sklearn, matplotlib

assert sys.version_info[:2] == (3, 13), sys.version
print("setup OK - Python", sys.version.split()[0])
'@ | python -
```

macOS ou Linux :

```
python3.13 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python - <<'PY'
import sys
import numpy, pandas, scipy, sklearn, matplotlib

assert sys.version_info[:2] == (3, 13), sys.version
print("setup OK - Python", sys.version.split()[0])
PY
```

À retenir

Toutes les commandes du TP sont écrites `uv run <commande>`. En voie pip/venv, retirez simplement le préfixe `uv run` (avec le venv activé) : `python scripts/...`, `pytest`.

Vérifiez l'arborescence :

```
tp_autonome_drift/
├── data/                  # référence + 3 semaines de production (labels à part, plus tard)
├── scripts/               # train_model  · drift_lab  · evaluate_semaine  · bonus_evidently
├── tests/test_drift.py    # votre definition of done
├── models/  reports/      # seront remplis par vos scripts
└── TP_apprenant_payguard_drift.md
```

Étape 1 - (Re)poser la baseline : entraîner et geler le modèle

Le script d'entraînement est fourni : il rejoue en accéléré votre Sprint 2 (split stratifié, HistGradientBoosting, métriques de validation, choix du seuil par le coût), puis gèle modèle et seuil.

```
uv run python scripts/train_model.py
```

Notez soigneusement les valeurs affichées (elles resservent à chaque étape) : PR-AUC, ROC-AUC, seuil gelé, rappel, précision, taux d'alerte. Puis lisez le script (10 lignes utiles) et `models/threshold.json` - c'est la « carte d'identité » du modèle, cousin de votre model card du module 22.

Questions d'observation - révision Sprint 2 (répondez par écrit, 3-4 phrases chacune) :

▶ Q1 (m11). La fraude pèse ~4,5 % des transactions. Pourquoi suit-on la PR-AUC plutôt que l'accuracy ? Que vaut la ligne de base d'une PR-AUC (le score d'un modèle aléatoire), et pourquoi n'est-ce pas 0,5 comme en ROC ?

▶ Q2 (m21). Le seuil gelé vous paraîtra très bas. Reliez-le au ratio de coûts FN/FP (50 € vs 2 €). Sur quel jeu ce seuil a-t-il été choisi, et pourquoi serait-il malhonnête de le (re)choisir plus tard sur les semaines de production ?

▶ Q3 (m11). Au seuil gelé, la validation montre ~28 % de taux d'alerte pour ~4,5 % de fraude réelle. L'équipe risque doit-elle vérifier ~850 dossiers par semaine de 3 000 transactions. Ce compromis précision/rappel vous semble-t-il défendable ? Avec quelle information métier trancheriez-vous ?

Étape 2 - Implémenter PSI et KS

Ouvrez `scripts/drift_lab.py`. Trois fonctions sont à compléter (les consignes détaillées et les indices sont dans les docstrings) :

1. `psi(ref, cur, bins=10)` - le Population Stability Index du cours : `PSI = Σ (p_cur - p_ref) · ln(p_cur / p_ref)`, bins calculés sur la référence, `+ 1e-6` contre les bins vides. ⚠ Réfléchissez au piège signalé en docstring : que fait `np.histogram` d'une valeur courante hors de la plage de la référence ?
2. `ks_pvalue(ref, cur)` - p-value du test de Kolmogorov-Smirnov (`scipy.stats.ks_2samp`).
3. `drift_table(df_ref, df_cur, ...)` - assemble le tableau : une ligne par feature, colonnes `feature / psi / ks_pvalue / verdict`, triée par PSI décroissant.

Validez au fur et à mesure avec les tests unitaires (aucune donnée réelle nécessaire) :

```
uv run python -m pytest -k "psi or ks or verdict or table" -v
```

8 tests verts = votre labo de dérive est opérationnel. Un test vous résistera peut-être : `test_psi_compte_les_valeurs_hors_plage_de_reference` - c'est le piège de la docstring.

Q4 (m31/m21). Vos deux outils ne racontent pas la même chose : que quantifie le PSI que la p-value KS ne dit pas, et que teste KS que le PSI ne dit pas ? Dans quel cas typique (pensez taille d'échantillon) KS crie-t-il « significatif ! » pour une dérive minuscule sans intérêt métier ?

Étape 3 - Semaine 1 : première ronde de surveillance

Les labels n'existent pas encore : vous ne pouvez comparer que des distributions d'entrée (c'est tout l'intérêt du covariate monitoring - module 31 §2.1).

```
uv run python scripts/drift_lab.py --semaine 1
```

Examinez la table et au moins deux figures dans `reports/figures/` (par exemple `drift_s1_montant_eur.png`).

Questions d'observation :

▶ Q5. Relevez le PSI max et les verdicts. Que concluez-vous pour la semaine 1, et quel « bulletin » envoyez-vous à l'équipe risque ?

▶ Q6. Les p-values KS ne valent pas toutes ~1, alors que rien n'a changé. Est-ce anormal ? (Que vaut une p-value sous H□, en espérance ?) Pourquoi ne faut-il PAS déclencher une alerte dès que p < 0,05 quand on teste 5 features chaque semaine ?

Étape 4 - Semaine 2 : quelque chose a changé

```
uv run python scripts/drift_lab.py --semaine 2
```

Contexte fourni par le métier (comme dans la vraie vie, il arrive APRÈS vos chiffres) : le site a lancé une « Vente Flash de rentrée » cette semaine-là, à grand renfort de publicité.

Questions d'observation :

▶ Q7. Classez les 5 features par PSI. Lesquelles franchissent 0,25 ? 0,10 ? Reliez chaque dérive forte à une explication métier plausible de la Vente Flash (regardez les figures !).

▶ Q8 (le cas `heure`). Pour `heure`, PSI et KS semblent se contredire. Lequel croire, pour quelle décision ? (Repensez à Q4 : ampleur vs significativité.)

▶ Q9. Une des features en dérive forte contribue très peu au score du modèle. Faut-il quand même la surveiller et remonter l'alerte ? Donnez un argument pour et un argument contre.

▶ Q10 (décision). PSI en dérive forte sur plusieurs features mais aucun label avant des semaines : que décidez-vous là, maintenant ? (Réentraîner tout de suite ? Attendre ? Surveiller un indicateur intermédiaire ? Prévenir qui ?) Justifiez le coût de chaque option.

Étape 5 - Semaine 3 : la ronde tranquille… ?

```
uv run python scripts/drift_lab.py --semaine 3
```

Questions d'observation :

▶ Q11. Que dit le PSI ? Rédigez le « bulletin » que vous enverriez à l'équipe risque sur la seule foi de ces chiffres.

▶ Q12 (la question qui fâche). Votre surveillance des entrées est muette. Cela prouve-t-il que le modèle fonctionne bien cette semaine-là ? Qu'est-ce que votre dispositif actuel serait structurellement incapable de voir ? (Module 31 §2.1 : quelle forme de dérive ne se voit pas dans P(X) ?)

Étape 6 - Les labels arrivent : l'heure de vérité

Trois semaines ont passé : les fraudes confirmées tombent (`data/labels/`). Vous pouvez enfin mesurer ce que valait vraiment le modèle, semaine par semaine - avec le seuil gelé, celui de l'étape 1.

Complétez `scripts/evaluate_semaine.py` (le chargement est fourni ; à vous la décision au seuil, la matrice de confusion et les métriques - révision directe des modules 11 et 21). Puis :

```
uv run python scripts/evaluate_semaine.py --semaine 1
uv run python scripts/evaluate_semaine.py --semaine 2
uv run python scripts/evaluate_semaine.py --semaine 3
uv run python -m pytest      # tout doit être vert désormais
```

Rassemblez les trois bulletins (le script tient un journal : `reports/suivi_semaines.csv`).

Questions d'observation - le cœur du TP :

▶ Q13 (m11). Semaine par semaine : taux de fraude réel, taux d'alerte, rappel, précision. Faites le lien avec vos verdicts PSI des étapes 3-5 : pour chaque semaine, le PSI avait-il « prédit » ce que vous mesurez maintenant ?

▶ Q14 (m11). Sur la semaine 3, comparez le mouvement de l'accuracy à celui du rappel. Expliquez, matrice de confusion à l'appui, pourquoi l'accuracy est structurellement incapable de raconter cette histoire sur une classe rare. (« Toujours pas de panne », version fraude…)

▶ Q15 (m21). Que vaut la ROC-AUC en semaine 3 ? Que signifie une AUC inférieure à 0,5 - et qu'est-ce que ça révèle sur ce que le modèle a « appris » du monde d'avant ? (L'AUC ne dépend pas du seuil : qu'est-ce que ça exclut comme explication ?)

▶ Q16 (synthèse module 31). Renseignez les trois fiches ci-dessous, puis nommez le type de dérive de chaque semaine (covariate drift / concept drift / aucun) en justifiant par deux preuves chacune (une côté distributions, une côté métriques métier) :

Chaque fiche garde les mêmes cinq rubriques :

- Semaine 1
- PSI des entrées :
- rappel :
- taux d'alerte :
- type de dérive :
- deux preuves : une distributionnelle et une métier.
- Semaine 2
- PSI des entrées :
- rappel :
- taux d'alerte :
- type de dérive :
- deux preuves : une distributionnelle et une métier.
- Semaine 3
- PSI des entrées :
- rappel :
- taux d'alerte :
- type de dérive :
- deux preuves : une distributionnelle et une métier.

▶ Q17 (KPI technique vs KPI métier). La semaine 2 et la semaine 3 sont deux pannes de nature opposée : l'une se voit sans labels, l'autre pas. Laquelle est la plus dangereuse en production, et pourquoi ? Quel « signal intermédiaire », disponible sans attendre les labels officiels, aurait pu vous mettre la puce à l'oreille en semaine 3 ? (Indice : l'équipe risque vérifie ~800 dossiers par semaine - que découvre-t-elle dans ses dossiers dès les premiers jours ?)

▶ Q18 (m21/m22). Pour chaque semaine 2 et 3, que déclenchez-vous : recalibrage ? changement de seuil ? réentraînement ? nouvelle référence ? mise à jour de la model card et des tests de non-régression ? Précisez l'ordre des actions et ce qui serait une erreur (par exemple : pourquoi « baisser le seuil » ne répare PAS la semaine 3 ?).

Étape 7 - Synthèse : votre drift spec

Le livrable final du module 31, transposé à PayGuard : créez `reports/drift_spec_payguard.md` à partir du gabarit ci-dessous et remplissez-le avec vos chiffres. Une drift spec fige QUI on compare, AVEC QUOI, À QUELLE FRÉQUENCE et QUAND on tire la sonnette - sans elle, le monitoring est arbitraire (module 31 §2.4).

En séance, exigez seulement six éléments : référence, fenêtre/fréquence, seuil, KPI métier, réaction et une limite connue. Les responsables, la matrice complète et les cas de repli constituent l'enrichissement autonome : ne retardez pas le dépôt de la preuve minimale pour les compléter.

```
# Drift spec - PayGuard v1

## 1. Référence
- Jeu de référence : … (quoi ? figé quand ? à quelle condition le remplacer ?)

## 2. Fenêtre courante & fréquence
- Fenêtre : … · Fréquence de calcul : … · Taille minimale d'échantillon : …

## 3. Features surveillées & tests
| Feature | Test(s) | Seuil d'alerte | Justification |
|---|---|---|---|
| montant_eur | | | |
| heure | | | |
| anciennete_client_j | | | |
| nb_articles | | | |
| distance_domicile_km | | | |

## 4. KPI métier (second rideau, dès que les labels tombent)
- KPI : … · Plancher déclencheur : … · Délai de disponibilité : …

## 5. Réactions au franchissement
- PSI > seuil sur ≥ N features → …
- Rappel < plancher → …
- Qui est prévenu, par quel canal, avec quel délai ?

## 6. Limites connues du dispositif
- … (au moins deux - la semaine 3 doit vous en inspirer une)
```

Definition of done du TP :

- [ ] `uv run python -m pytest` : 12 tests verts, 0 échec
- [ ] `reports/` contient : 3 tables de dérive, les figures, `suivi_semaines.csv`, `drift_spec_payguard.md`
- [ ] Réponses écrites Q1 → Q18 (elles seront discutées, pas notées à la virgule)

Étape 8 - BONUS : le même audit avec Evidently (sur consigne du formateur)

Evidently (même famille de version que le lock InduSense) emballe ce que vous venez de coder à la main dans un rapport HTML - c'est l'outil du module 32.

```
uv sync --frozen --extra dev --extra evidently      # (pip : pip install "evidently>=0.7,<0.8")
uv run python scripts/bonus_evidently.py --semaine 2
uv run python scripts/bonus_evidently.py --semaine 3
```

Ouvrez les HTML générés dans `reports/` et comparez à vos tables : mêmes features signalées en semaine 2 ? même silence en semaine 3 ? quels tests Evidently a-t-il choisis par feature, et pourquoi peut-il différer de vos PSI à vous (binning, seuils par défaut) ? *Si l'installation coince : ce bonus est sans enjeu, votre `drift_lab.py` fait déjà le travail - c'est le « repli maison » officiel du module 32.*

Q-bonus. Votre drift spec (étape 7) survivrait-elle au passage à Evidently ? Qu'est-ce qui relève de l'outil (remplaçable) et qu'est-ce qui relève de la spec (vos décisions) ?

Étape 9 - Passage vers l'observabilité M33-M34

Ne lancez pas d'exporteur ni de stack Prometheus/Grafana dans ce TP : cette dépendance n'est pas scellée dans son `uv.lock`. Conservez vos CSV et votre drift spec ; ils seront reliés à l'exporteur, aux SLO, au dashboard et aux alertes dans les modules 33-34 sur le repo InduSense.

À retenir

[OK] Preuve de transition : être capable de nommer un KPI technique (PSI/KS), un KPI métier (rappel/précision/taux d'alerte), le seuil associé et l'action prévue. Aucun Docker n'est requis ici. Suite au module 32 : ce calcul devient un rapport produit à chaque run dans le flow Prefect, après `predict`, avec alerte anti-bruit (seuil et cooldown) et événements en base. Vos choix de drift spec - features, tests, seuils, fenêtre et fréquence - restent inchangés : l'outil change, pas la spec.